

Relatório do Sistema

Conciliador OFX × Planilha × Domínio — funcionalidades implementadas (Janco Assessoria Contábil)

1. Visão geral

Aplicativo desktop em Python/Tkinter para conciliação em 3 níveis e classificação contábil automática. Multi-usuário (rede) com autenticação individual, dados persistidos em MariaDB compartilhado.

- **Planilha (.xlsx)** com contas a pagar — só Vencimento e Valor são obrigatórios; demais campos (pagamento, emissão, NF, CNPJ, fornecedor, histórico, tipo) são opcionais.
- **Extrato bancário (OFX)**: aceita **múltiplos arquivos** de bancos diferentes; identifica o banco de cada transação.
- **Sistema Domínio (Escrita Fiscal + Contábil)** via ODBC (read-only): puxa parcelas, status (Aberto/Parcial/Paga) e o plano de contas da empresa (filtrado só por contas analíticas).
- **Lançamentos contábeis automáticos**: pendentes do OFX, pares que faltam no Domínio E pendentes da planilha (Caixa geral) podem ser classificados por regras salvas por empresa ou manualmente.
- **Comparação OFX × Domínio direta**: mesmo sem planilha, dá pra conferir se os pagamentos do extrato bateram com o Domínio.
- **Multi-usuário com login individual**: cada operador tem seu próprio username/senha; empresa ativa é individual; regras e mapeamentos são compartilhados.

2. Arquitetura

Stack tecnológico

- **Python 3.11+ + Tkinter** (UI desktop nativa)
- **MariaDB 10.5+** como banco de configurações compartilhado (via PyMySQL, pure Python — sem compilação C)
- **SQL Anywhere (Domínio Contábil)** via **pyodbc** (read-only)
- **openpyxl** para .xlsx (leitura e escrita), **ofxparse** para OFX
- **reportlab** para geração do PDF de documentação
- **PyInstaller** para empacotar em .exe distribuível (Windows)

Modelo de deployment

- **Um servidor MariaDB** na rede local (ex: 10.0.1.47:3307) guardando todas as configurações, regras, mapeamentos e usuários.
- **Máquinas cliente** rodam o Conciliador.exe empacotado, conectando no MariaDB via rede.
- **Domínio Contábil** continua sendo lido diretamente via ODBC em cada máquina (não é feito cache no MariaDB).
- **Distribuição**: ZIP com .exe + pasta data/ (credenciais do MariaDB já preenchidas). Cada operador só configura o *dominio_config.json* com o próprio login do Domínio.

3. Autenticação e usuários

Login individual

- Tela de login aparece antes da UI principal, forçada ao topo (-topmost + focus_force pra não ficar atrás de outras janelas).

- Autenticação por username + senha, validados contra a tabela *usuario* do MariaDB.
- Senhas armazenadas com **PBKDF2-HMAC-SHA256**, 200.000 iterações, salt aleatório de 16 bytes. Formato: *iter\$salt\$hash*. Comparação em tempo constante (`hmac.compare_digest`).
- Após login: *usuario.ultimo_login* atualizado; *config.set_usuario_atual(id)* registra pra config saber de qual usuário é a *empresa_ativa*.

Perfis

- **admin**: gerencia (criar/editar/desativar) outros usuários, vê botões *Fonte: pagamentos*, *Fonte: plano contas*, *Gerenciar usuários*.
- **operador**: uso normal do app + trocar a própria senha. Botões de configuração escondidos.

Empresa ativa por usuário

- Antes: *dominio_empresa* era global no *config.json*.
- Agora: coluna *usuario_empresa_ativa* (JSON) — cada login tem sua própria empresa selecionada.
- **Regras e mapeamentos continuam por empresa** (compartilhados) — A muda regra da empresa 55, B vê o mesmo estado.
- **Fontes SQL do Domínio** continuam globais (mesma config pra todos, faz sentido — SQL de acesso ao Domínio é do sistema).

Barra do topo

Todo usuário vê 'Usuário logado — perfil' + botões 'Minha senha' e 'Trocar usuário'. Admin vê também 'Gerenciar usuários'.

4. Leitura de dados

Planilha Excel (.xlsx) — `parser_xlsx.py`

- Detecção automática da linha do cabeçalho (busca nas primeiras 15).
- Auto-detecção dos campos por nome, com dezenas de aliases pra cada (Vencimento, Pagamento, Emissão, Valor, NF, CNPJ, Fornecedor, Histórico, Tipo).
- Fallback por conteúdo (datas → Data, números → Valor, textos longos → Fornecedor).
- **Apenas Vencimento e Valor são obrigatórios.** No diálogo de mapeamento cada combo tem opção (*deixar vazia*).
- Preview ao vivo (10 linhas) com marcação vermelha em linhas inválidas.
- Conversão BR de valores (R\$ 1.234,56, parênteses pra negativo).

Extrato OFX — `parser_ofx.py`

- Leitura via `ofxparse`. Aceita **múltiplos arquivos** num único Importar (Ctrl+clique).
- Identifica o **banco** de cada lançamento via OFX (`institution.organization`, `routing_number`, `account_id`) com fallback ao nome do arquivo.
- Extrai o número do **documento** (CHECKNUM ou FITID) — usado em regras tipo memo.
- Filtra automaticamente só pagamentos (valores negativos), invertendo o sinal pra casar com a planilha.

Comprovantes PDF de pagamento — `parser_pdf.py`

- Extração **nativa** de PDFs (via `pdfplumber`) de comprovantes de pagamento de boleto. Não requer OCR — funciona só em PDFs digitais (baixados de internet banking).
- **Bancos suportados hoje:** Sicoob (SISBR) e Bradesco NET Empresa. Arquitetura permite adicionar novos bancos com detecção automática por marcadores fortes no texto.
- **Campos extraídos:** valor pago, data pagamento, data vencimento, beneficiário (nome e CNPJ), nº documento, banco.
- **Multi-comprovante:** um único PDF pode ter dezenas ou centenas de comprovantes empilhados — o parser divide por cabeçalhos e processa cada um separadamente.
- **Streaming pra PDFs grandes:** performance calibrada pra arquivos de 500+ páginas (~50 ms/pág). UI mostra progresso.
- **Deduplicação cruzada:** se a planilha já tem esse lançamento, detecta e ignora (ver seção 5).

Sistema Domínio (ODBC) — `parser_dominio.py`

- **Auto-conexão no startup:** usa credenciais salvas em `data/dominio_config.json`. Silencioso se falhar — usuário pode reconectar manualmente.
- **Duas fontes independentes** (`pagamentos` e `plano_contas`) — cada uma com seu SQL e mapeamento próprios.
- `extrair_pagamentos` detecta colunas opcionais `status_parcela` e `valor_pago`.
- `extrair_plano_contas` **filtra automaticamente só contas analíticas** (tipo A). Reconhece a coluna de tipo pelo nome (TIPO_CTA, CTRG_CTA, etc.) ou mapeamento explícito.
- Injeção automática de `CODI_EMP = ?` quando o SQL tem `?`.
- `listar_filiais(conn, cnpj_matriz)` descobre empresas do mesmo grupo via **CNPJ raiz** (primeiros 8 dígitos): útil quando a matriz paga boletos das filiais.
- `encontrar_matriz(conn, cnpj)` identifica a matriz do grupo pelo sufixo **/0001-XX** do CNPJ (regra Receita Federal). Fallback: menor `codi_emp` do grupo.

SQL recomendado — plano de contas

```
SELECT CLAS_CTA, NOME_CTA, TIPO_CTA
FROM bethadba.ctcontas
WHERE CODI_EMP = ? AND TIPO_CTA = 'A'
ORDER BY CLAS_CTA
```

5. Lógica de conciliação

Nível 1 — Planilha × OFX (matcher.py)

- **conciliar_automatico** em DUAS sub-fases:
- **1.1 Match exato** por (data_pagamento se mapeada senão vencimento, valor). Cada Transacao do OFX só casa com 1 planilha.
- **1.2 Match aproximado pequeno**: nos restantes, $\Delta \text{dias} \leq 2$ E $\Delta \text{valor} \leq \text{R\$ } 0,10$. Caem em Conciliados (com diferença visível).
- **gerar_sugestoes**: $\Delta \text{dias} \leq 2$ e $\Delta \text{valor} \leq \text{R\$ } 10$ — ficam na aba Sugestões com seleção múltipla e 'Aceitar tudo'.

Deduplicação Planilha × Comprovantes PDF

- Antes da conciliação, se o usuário importar planilha .xlsx E comprovantes PDF, o app detecta lançamentos que aparecem nos dois pra evitar duplicidade.
- **Regra de match**: valor exato + data (vencimento OU pagamento) coincidem + identidade (CNPJ igual OU nome bate por substring normalizada, sem sufixos societários).
- Quando detecta duplicata: mantém a linha da planilha e usa o PDF só pra **enriquecer** campos faltantes (ex: se a planilha não tem CNPJ, herda do PDF).
- Nunca considera duplicata se ambos lados estão sem CNPJ e sem nome (evita colar tarifas iguais em datas iguais como se fossem uma só).
- Popup pós-importação lista quantos novos entraram e quantos duplicados foram ignorados, pra visibilidade.

Nível 2 — Comparação com Domínio (4 fases)

- **_filtrar_conciliados_por dominio** processa **3 fontes** (pares P×OFX, pendentes planilha, pendentes OFX) em **4 fases** hierárquicas. Cada Transacao do Domínio só casa com 1 item.
- **Fase 1 — Exata**: data_vencimento + valor + NF iguais.
- **Fase 2 — 2 de 3**: pelo menos 2 dentre (CNPJ, data, valor) iguais. O campo restante pode divergir; a diferença aparece na coluna Δ Domínio.
- **Fase 3 — Fornecedor + Valor**: valor exato E (CNPJ bate OU nome bate por substring normalizada). Data usada apenas como desempate — NÃO precisa bater. Útil quando o Domínio tem a mesma parcela mas com vencimento renegociado/prorrogado.
- **Fase 4 — NF + Fornecedor (valor livre)**: Nº NF normalizado igual (obrigatório e não-vazio dos dois lados) E (CNPJ bate OU nome bate). Valor **NÃO** precisa bater — cobre pagamentos com juros/multa/desconto onde o valor divergiu da parcela original mas a NF é a mesma. Data como desempate. Δ dias e Δ valor ficam visíveis na coluna Δ Domínio.
- Prioridade nas 4 fases: pares > pendentes planilha > pendentes OFX.

Comparação OFX × Domínio direta (sem planilha)

- Cenário útil quando o operador só tem OFX e Domínio (não recebeu planilha de contas a pagar).
- Botão 'Conciliar' habilita com planilha OU OFX (não exige os dois).
- Match OFX × Domínio segue as mesmas 4 fases.
- Pendentes OFX que casarem com Domínio somem da aba Pendentes e aparecem em Conciliados × Domínio com 'Origem = banco do OFX'.

Enriquecimento cruzado — PDF → OFX → Domínio

- Após conciliar, se um par tem *planilha.origem* = 'pdf', o app copia beneficiário/CNPJ/nº doc do PDF para o extras do OFX.
- Efeito visual: aba OFX (dados crus) mostra colunas Fornecedor e CNPJ populadas + linhas com fundo azul claro. Aba Pendentes lado OFX também mostra dados enriquecidos, útil pra criar regras memo.
- Efeito funcional: regras memo automaticamente usam o fornecedor enriquecido como padrão de match; lançamentos manuais herdam CNPJ/fornecedor sem digitação.
- **Aba Conciliados × Domínio:** dados priorizam sempre o Domínio > planilha/PDF > OFX. Se o CNPJ da planilha está errado ou vazio, puxa o do Domínio (que é a fonte contábil confiável).

Preservação seletiva ao limpar

- Ao clicar em **Limpar planilha**: mantém intactos pares conciliados, matches com Domínio, lançamentos contábeis E pendentes OFX que ainda estão carregados.
- Ao clicar em **Limpar OFX**: idem, preservando pendentes da planilha.
- **Reconciliação seguinte não duplica:** linhas já pareadas não são reoferecidas — só o que sobrou tenta casar com o novo OFX/planilha importado.
- Trocar de empresa continua fazendo **reset total** (dados são específicos por empresa).

Grupo matriz + filiais — carregamento consolidado

- **Problema real:** matriz frequentemente paga boletos emitidos contra filiais. Se o app só puxa parcelas do *codi_emp* da matriz, as notas ficam órfãs (amarelo — 'falta no Domínio') sem motivo.
- **Deteção automática:** no clique de 'Carregar pagamentos', *listar_filiais* compara o CNPJ raiz (8 primeiros dígitos) da matriz com todas as empresas cadastradas no Domínio e retorna as que casam.
- **Carregamento:** chama *extrair_pagamentos* uma vez por *codi_emp* encontrado. Cada Transacao recebe *extras['codi_emp_origem']* e *extras['razao_empresa']*, então dá pra identificar de qual filial veio a parcela.
- **Interface:** aba *Domínio dados* ganha coluna '**Empresa (código)**'. Título da aba mostra '*Domínio dados (N | X empresas)*' quando grupo detectado. Ao carregar, aparece dialog listando as empresas incluídas.
- **Fallback:** sem CNPJ da matriz, sem raiz válida, ou grupo com só 1 empresa, comportamento é idêntico ao anterior.
- **Plano de contas** continua vindo **só da matriz**, como definido pela regra contábil. Se o operador seleciona uma filial no combo de empresa, o app usa *encontrar_matriz* pra achar o *codi_emp* da matriz (sufixo /0001) e chama *extrair_plano_contas* com esse código — o messagebox informa qual matriz foi usada.
- **Regras de taxa por empresa** continuam separadas: cada empresa do grupo tem suas próprias regras salvas em *regra_taxa.codi_emp*. Isso é intencional — filiais têm contas bancárias diferentes, então as regras que apontam pra essas contas ficam por empresa.

6. Lançamentos contábeis

6 tipos de lançamento cobrindo todas as combinações origem (par/pendente) × modo (regra/manual):

tipo_regra	Origem	Banco	Notas
memo	Regra automática casa contra memo + documento do OFX	Do OFX	Tarifas, IOF, juros. Regra tem campo opcional Banco para evitar falso positivo entre bancos diferentes.
fornecedor	Regra automática casa contra par P × OFX sem Domínio	Do OFX (banco do par)	Casa contra CNPJ, fornecedor, histórico OU tipo da planilha.
fornecedor_planilha	Regra automática casa contra pendente da planilha (sem OFX)	Caixa geral	Mesma regra fornecedor, aplicada ao pendente-só-planilha.
manual	Lançamento manual a partir de par P × OFX (aba Comparação)	Do OFX	Botão 'Lançar manualmente' — não cria regra.
manual_ofx	Lançamento manual a partir de pendente OFX	Do OFX	Botão na aba Pendentes lado OFX.
manual_planilha	Lançamento manual a partir de pendente da planilha	Caixa geral	Botão na aba Pendentes lado Planilha.

Regras automáticas — configuração

- Botão **Configurar taxas** abre diálogo com lista editável.
- 2 botões: + *Regra por memo* e + *Regra por fornecedor*.
- Regras **memo** podem ter campo opcional *Banco* — se preenchido, regra só dispara em transações daquele banco (evita falso positivo entre bancos com memos parecidos).
- Regras **fornecedor** casam contra CNPJ + nome + histórico + TIPO da planilha (substring case-insensitive).
- Campo Conta é Combobox autocomplete carregando plano de contas filtrado por analíticas.
- Regras salvas por empresa em *regra_taxa.codi_emp*.
- **Atalhos contextuais na aba Pendentes**: pré-populam a regra com dados da linha selecionada.
- **Histórico contábil é opcional**: quando o campo Histórico da regra fica em branco, o lançamento gerado herda o texto bruto da origem — *memo do OFX* para regras **memo**, *histórico da planilha* para regras **fornecedor**. Implementado em *lancamentos._historico_final*. Útil quando cada linha tem descritivo próprio e não faz sentido padronizar.

Editar e excluir lançamentos

- Aba **Lançamentos contábeis** tem botões **Editar lançamento** e **Excluir lançamento**.
- **Editar**: diálogo com data/valor/banco/conta/histórico editáveis. Se lançamento veio de regra, é 'promovido a manual' — a versão editada persiste, transação origem é marcada como ignorada.
- **Excluir**: manual remove; automático marca origem como ignorada. Transação volta pra aba Pendentes.

7. Aba Comparação — 6 cores

Cabeçalho tem legenda com 6 chips coloridos indicando o significado de cada cor:

- ☐ **OK** (verde) — Conciliado P × OFX e no Domínio
- ☐ **Falta dom** (amarelo) — Conciliado P × OFX, falta no Domínio
- ☐ **Caixa OK** (azul claro) — Pendente planilha (Caixa geral) no Domínio
- ☐ **Caixa falta** (cinza) — Pendente planilha, falta no Domínio
- ☐ **OFX OK** (ciano) — Pendente OFX (sem planilha) no Domínio
- ☐ **OFX falta** (laranja) — Pendente OFX, falta no Domínio

Título da aba mostra contagem por grupo. Ações (Editar dados, Lançar manualmente, Criar regra, Exportar pendências) funcionam em qualquer linha 'falta'.

8. Interface gráfica

Barras superiores (4 blocos)

- **Usuário:** logado, perfil, botões Minha senha / Gerenciar / Trocar.
- **Domínio:** Conectar, Selecionar empresa, Fonte pagamentos (admin), Fonte plano contas (admin), Carregar pagamentos, Carregar plano contas.
- **Planilha:** Abrir, Editar colunas, Limpar planilha.
- **OFX:** Importar, Limpar OFX.
- **Ações:** Conciliar, Comparar com Domínio, Configurar taxas.

Filtros das abas de dados

- **Busca global** (Buscar:) filtra em tempo real.
- **Status** (Domínio dados): dropdown Todos/Aberto/Parcial/Paga.
- **Filtros estilo Excel por coluna:** clique no cabeçalho ☐ → popup com checkboxes + busca + marcar/desmarcar tudo.
- Cabeçalho mostra ▼ ☐ em colunas com filtro ativo.

Abas do Notebook (10 abas)

#	Aba	Conteúdo
0	Planilha	Dados crus da planilha .xlsx OU dos comprovantes PDF (Sicoob/Bradesco). Botões Abrir planilha e Importar comprovantes PDF. Deduplica automaticamente entre xlsx+PDF.
1	OFX	Pagamentos do extrato (data, banco, valor, memo, documento). Multi-arquivo. Após conciliação, ganha colunas Fornecedor (via PDF) + CNPJ (via PDF) + fundo azul nas linhas enriquecidas.
2	Domínio dados	Parcelas do Domínio com status (Aberto/Parcial/Paga).
3	Conciliados	Pares Planilha × OFX casados. Coluna 'Origem' mostra o banco do OFX.
4	Pendentes	Layout vertical (planilha em cima, OFX embaixo). 4 botões: Lançamento manual e Criar regra para cada lado. Botão Exportar Excel.
5	Sugestões	Pares aproximados ($\Delta \leq 2d$ e $\Delta \leq R\$ 10$). Seleção múltipla.
6	Conciliados × Domínio	Pares triple-matched + Caixa geral + OFX (sem planilha) que casaram no Domínio. Botão Exportar Excel.
7	Comparação	6 status coloridos (ok, falta dom, caixa ok/falta, OFX ok/falta). Legenda + 4 botões (Editar dados, Lançar manual, Criar regra, Exportar pendências).

8	Lançamentos contábeis	Saídas geradas por regras ou manualmente (6 tipos). Botões Editar, Excluir, Exportar Excel.
9	Plano de contas	Plano da empresa carregado do Domínio, FILTRADO só por contas analíticas (tipo A).

9. Persistência (MariaDB)

Banco compartilhado

- Todas as configurações do sistema, regras, mapeamentos e usuários ficam em MariaDB compartilhado na rede local.
- Cada máquina cliente lê/escreve diretamente no banco — não há backend intermediário.
- Camada de acesso: *db.py* (PyMySQL + context manager) e *config.py* (API carregar/salvar compatível com o *config.json* antigo, agora sobre MariaDB).

Schema (4 tabelas)

Tabela	Conteúdo	Colunas principais
app_config	Configurações globais (fontes SQL do Domínio, chaves/valores JSON)	chave (PK), valor (JSON), atualizado_em
regra_taxa	Regras de classificação contábil, uma linha por regra	id (PK), codi_emp, tipo (memo/fornecedor), padrao, historico, conta, banco, ordem
mapeamento_planilha	Mapeamento coluna→campo por empresa	id (PK), codi_emp, campo, nome_coluna (UNIQUE codi_emp+campo)
usuario	Usuários do app com senha (PBKDF2)	id (PK), username (UNIQUE), nome, email, senha_hash, ativo, admin, empresa_ativa (JSON), ultimo_login, criado_em

Setup e migração

- *setup_db.py*: script interativo — pede credenciais MariaDB, cria banco + tabelas, cadastra 1º admin, migra *config.json* antigo se houver.
- Idempotente (CREATE IF NOT EXISTS); pode rodar de novo sem risco.
- Bootstrap: se não existe nenhum usuário, força criação do 1º admin no setup.

Credenciais

- *data/db_config.json*: host, porta, user, senha, database.
- *data/dominio_config.json*: DSN, usuário, senha do Domínio (individual por operador — cada consulta ODBC fica rastreada no Domínio).
- Ambos são gitignored — nunca vão pro repositório.

10. Exportação para Excel (.xlsx)

3 abas do app têm botão 'Exportar para Excel' — geram .xlsx com estilo consistente (cabeçalho azul, freeze pane na linha 1, cores de status preservadas):

- **Aba Pendentes**: gera .xlsx com 2 abas (Pendentes Planilha + Pendentes OFX) — colunas específicas de cada lado.
- **Aba Conciliados × Domínio**: pares triple-matched + Caixa geral + OFX (todos os matchados). Coluna Origem indica banco ou 'Caixa geral'.
- **Aba Lançamentos contábeis**: todos os lançamentos com tipo_regra legível + linha TOTAL com fórmula =SUM() no fim.
- **Aba Comparação**: botão Exportar pendências separa em 3 abas por tipo (Amarelos P × OFX / Cinzas Caixa geral / Laranjas OFX-só). Cores das linhas preservadas.

11. Distribuição (build .exe)

Empacotamento via PyInstaller (spec *Conciliador.spec*). Modo onedir — gera pasta *dist/Conciliador/* com:

- **Conciliador.exe** — entry point (6 MB)
- **_internal/** — libs Python + DLLs (pyodbc, tkinter, MariaDB driver, reportlab, etc.)
- **data/db_config.json** — apontando pro servidor MariaDB (já preenchido, distribuído junto).
- **data/dominio_config.json.EXEMPLO** — template pro operador preencher com seu login individual do Domínio.
- **LEIA-ME.txt** — instruções básicas.
- Zipado (~25 MB) e distribuído por pen drive, rede ou email.

Detecção runtime do modo empacotado

db.py e *parser_dominio.py* têm helper *_base_dir()* que detecta *sys.frozen* (PyInstaller) e usa o diretório do executável em vez de *__file__.parent*. Assim as credenciais ficam ao lado do .exe (editáveis) e não enterradas em *_internal*.

12. Diagnóstico ODBC (testar_dominio.py)

- Script standalone que testa a conexão ODBC com o Domínio em 6 passos, identificando exatamente onde trava:
- 1. pyodbc carregado? 2. dominio_config.json existe e é JSON válido? 3. DSN listada no ODBC 64-bit? 4. Conexão abre? 5. SELECT roda? 6. Tem permissão em bethadba?
- Detecta erros específicos: IM014 (incompatibilidade 32/64 bit), 28000 (credenciais inválidas), 08001 (servidor Domínio fora).
- Empacotável como .exe standalone pra distribuir aos operadores quando derem problema.

13. Convenções herdadas do projeto Janco

- Tabelas do Domínio Escrita Fiscal e Contábil vivem no schema **bethadba** (sempre prefixar).
- Conexão ODBC é sempre **read-only** — escrita exigiria RPA (fora do escopo deste app).
- **Gotcha SQL Anywhere**: DISTINCT + ORDER BY exige aliases do SELECT (não nomes qualificados — erro -854).
- Empresas compartilham a mesma base — separação por **CODI_EMP** em cada tabela; o app injeta o filtro automaticamente quando o SQL tem ?.

14. Tabelas do Domínio utilizadas

- **bethadba.efentradas** — cabeçalho de NF de entrada.
- **bethadba.efentradaspar** — parcelas/duplicatas (vcto, valor, número da parcela).
- **bethadba.efentradaspag** — pagamentos registrados (usado pra calcular status agregado da parcela).
- **bethadba.eforenece** — cadastro de fornecedores.
- **bethadba.geempre** — cadastro de empresas.
- **bethadba.ctcontas** — plano de contas (CLAS_CTA, NOME_CTA, TIPO_CTA) filtrado por CODI_EMP e TIPO_CTA = 'A'.

15. Infraestrutura e código-fonte

- **requirements.txt**: openpyxl, ofxparse, pyodbc, reportlab, PyMySQL.
- **.gitignore**: protege credenciais (db_config, dominio_config), config.json legado, backups .sql, dist/, build/, .exe, planilhas, OFX, CSV.
- **iniciar.bat**: git pull + venv + pip install + roda setup_db se primeira execução + abre app.
- **Conciliador.spec**: config do PyInstaller com hidden imports declarados e pasta data/ incluída.
- **gerar_pdf.py**: regenera este relatório.
- **Repositório**: github.com/Fernandovini2607/conciliador (privado).

16. Documentação incluída no projeto

- **INSTALACAO_CLIENTE.md** — método Git (para desenvolvedores): Python + Git + venv + pip install. ~30 min.
- **INSTALACAO_CLIENTE_EXE.md** — método ZIP (para operadores): extrair, criar dominio_config.json, atalho. ~10 min. Recomendado.
- **relatorio_sistema.pdf** — este documento.

17. O que ainda não existe

- Tolerâncias configuráveis via UI (hoje fixas em código).
- Histórico de conciliações entre execuções (auditoria).
- Match por similaridade textual (fuzzy match) em fornecedor.
- Escrita de volta no Domínio (exigiria RPA via pywinauto).
- Edição dos dados do lado OFX (só a planilha é editável hoje).
- Tela de configuração de credenciais Domínio no primeiro login (hoje é manual editando JSON).
- Recuperação de senha por email.
- Auditoria de ações (quem fez o quê e quando).